

Scope

Scope

Dove esistono le variabili e perche alcune non sono visibili ovunque.

Cosa significa scope

Lo **scope** e la zona del programma in cui una variabile esiste ed e visibile.

Una variabile non vive ovunque. Di solito vive solo dentro il blocco in cui e stata creata.

Variabili locali

Una variabile creata dentro un metodo si chiama variabile locale.

```
public static void saluta() {  
    String nome = "Luca";  
    System.out.println(nome);  
}
```

`nome` esiste solo dentro `saluta` .

Questo codice e sbagliato:

```
public static void saluta() {  
    String nome = "Luca";  
}  
  
public static void main(String[] args) {  
    System.out.println(nome); // errore  
}
```

`main` non vede la variabile `nome` .

Parametri

Anche i parametri esistono solo dentro il metodo.

```
public static void saluta(String nome) {  
    System.out.println("Ciao, " + nome);  
}
```

`nome` è visibile dentro `saluta`, ma non fuori.

Se vuoi usare un valore in un metodo, passalo come parametro.

Blocchi con parentesi graffe

Anche un blocco `if` o `for` crea una zona.

```
if (true) {  
    int numero = 10;  
    System.out.println(numero);  
}  
  
// System.out.println(numero); // errore
```

`numero` nasce dentro il blocco `if` e sparisce alla fine del blocco.

Variabili create nel for

```
for (int i = 0; i < 3; i++) {  
    System.out.println(i);  
}  
  
// System.out.println(i); // errore
```

La variabile `i` esiste solo dentro il ciclo.

Questo è utile: evita che contatori temporanei restino in giro nel resto del programma.

Due variabili con lo stesso nome

Non puoi dichiarare due variabili con lo stesso nome nello stesso blocco.

```
int eta = 20;  
int eta = 21; // errore
```

Puoi invece modificare la variabile:

```
int eta = 20;  
eta = 21;
```

Campi di classe

Più avanti useremo variabili dichiarate dentro una classe, ma fuori dai metodi. Si chiamano **campi**.

```
public class Persona {  
    String nome;  
    int eta;  
}
```

Questi campi appartengono agli oggetti della classe `Persona`.

Per ora ti basta distinguere:

- variabili locali: dentro un metodo o blocco
- parametri: valori ricevuti da un metodo
- campi: dati di un oggetto

Esempio corretto con parametri

Se un metodo deve usare un valore, passalo:

```
public class EsempioScope {  
    public static void saluta(String nome) {  
        System.out.println("Ciao, " + nome);  
    }  
  
    public static void main(String[] args) {  
        String nomeUtente = "Sara";  
        saluta(nomeUtente);  
    }  
}
```

Output:

```
Ciao, Sara
```

`nomeUtente` esiste in `main`. Il suo valore viene passato a `saluta`, dove arriva nel parametro `nome`.

Regola pratica

Quando Java dice che una variabile "cannot be resolved", spesso significa che stai provando a usarla fuori dal suo scope.

Controlla dove è stata dichiarata e dentro quali parentesi graffe si trova.